Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- İkinci ödev gereksinimleri
  - Kullanıcı giriş ekranı ve kullanıcı kayıt ekranı gereksinimi; başarılı aksiyonlarda mesajlarla bilgilendirme.
  - Kayıt ekranından alınacak bilgiler, şifrelerin eşleşmemesi durumunda hata mesajı.
  - Aynı kullanıcı adının sistemde var olması durumunda hata mesajı.
  - Kullanıcı bilgilerini kalıcı alana taşıma: SQLite veya dosya yapısı (artık ArrayList yeterli değil).
  - Menü ekranı: soru ekle, soru listele, sınav oluştur, sınav ayarları gibi faaliyetler (farklı tasarımlar kabul edilir; tab, floating action button, action bar vb.).
  - Soru ekleme ekranı: 5 şık, doğru cevap, resim/ses/video eklenebilme.
  - Soru listeleme ekranı (RecyclerView): soruları silme, tıklayarak güncelleme; silme sırasında dialog box ile emin misiniz sorusu.
  - Sınav ayar ekranı: sınav süresi, soru puanı, zorluk düzeyi (2-5 arası) gibi bilgileri SharedPreferences ile saklama.
  - Zorluk düzeyi 2 olduğunda 2 şık (1 doğru, 1 yanlış), düzey 5 olduğunda 5 şık.
  - Sınav oluşturma ekranı: ayarları değiştirebilme, oluşturulan sınavı metin formatında kaydetme.
  - WhatsApp/Signal gibi mesajlaşma uygulaması üzerinden paylaşabilecek kod üretme.
  - Teslim: kaynak kodları + APK dosyası GitHub'a, 3-5 dakikalık YouTube videosu, PDF (link içeren) Online Yıldız'a.
  - UI puanı: tutarlı, dengeli, hızlıca her şey yerleştirilmiş olmayan, planlı arayüzler bekleniyor; yeni UI bileşenleri kullanmak ekstra puan.
- Veri saklama yöntemleri
  - Dört temel veri saklama yöntemi:
    1. App-specific storage (uygulamaya özel depolama): dahili ve harici bellek olmak üzere iki türlü.
    2. Shared storage: video, fotoğraf, ses, belge gibi paylaşılabilir içerikler.
    3. Preferences: uygulama ayarları, oyun ayarları gibi key-value formatında basit veriler.
    4. Veritabanı: yapılandırılmış, ilişkisel veri (SQLite + Room library, Firebase).
  - Lokalde vs network/cloud'da veri saklama seçenekleri.
- App-specific storage detayları
  - getFilesDir (kalıcı bilgiler) ve getCacheDir (geçici bilgiler) dizinleri.
  - Harici bellek karşılığı: getExternalFilesDir ve getExternalCacheDir.
  - Uygulama kaldırıldığında app-specific dosyalar otomatik olarak silinir.
  - Shared storage: MediaStore API (medya dosyaları) ve Storage Access Framework (diğer dosyalar).
  - İzinler: READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE, MANAGE_EXTERNAL_ST
  - Android 10 ve 11 ile gelen Scoped Storage; belli bölgelere erişim yeteneği kazandırma.
  - Android 10 ile gelen internal storage'daki tüm verilerin şifrelenmesi özelliği.
- SharedPreferences
  - Primitif verileri (int, long, float, boolean) ve string'i key-value şeklinde saklar.
  - Uygulama kaldırıldığında shared preferences dosyaları da silinir.
  - Diğer uygulamalar tarafından erişilemez (güvenli alan).

  - Tek bir veya birden fazla shared preferences dosyası oluşturulabilir.
  - getSharedPreferences veya Activity'nin kendi getPreferences metodu kullanılabilir.
- SQLite ve Room
  - Yapılandırılmış, ilişkisel veriler için internal storage'da veritabanı.
  - En alt katmanda SQLite, üst katmanda Room library ile erişim kolaylığı.
  - Room, SQLite'ın verimli kullanımını sağlar.
- Internal vs External storage
  - Internal storage sürekli erişilebilir; external storage fiziksel olarak orada olduğu sürece erişilebilir.
  - Internal storage'da harici alan yaratma imkanı vardır (fiziksel SD card olmak zorunda değil).
  - USB olarak Android telefon USB storage olarak enable edildiğinde external storage ile çalışan uygulamalar bloklanır ve kapatılır (telefon hard drive moduna geçer).
  - external storage'a yüklenen uygulamaların APK'sı orada olsa da, uygulamaya özel dosyalar internal storage'da bulunur.
  - Manifest'te installLocation özelliği ile uygulamanın external'a mı yoksa internal'a mı kurulacağı belirlenebilir (preferExternal, auto).
- Dosya okuma/yazma işlemleri
  - getFileStream: file objesi üzerinden dosya oluşturma.
  - openFileOutput metodu: context üzerinden doğrudan dosyaya erişim.
  - FileOutputStream (yazma), FileInputStream (okuma).
  - write, close, flash metotları.
  - flash: verilerin senkron olarak storage'a yazılmasını sağlar; çağrılmazsa asenkron olarak uygun zamanda yazılır.
  - Try-catch mekanizması: dosya yok, açılamaz gibi durumlar nedeniyle zorunlu.
  - StreamReader ile satır satır okuma.
- Cache kullanımı
  - Cache dizini: sonradan silinmesi sorun yaratmayacak, kalıcı olması gerekmeyen dosyalar için.
  - İnternetten indirilen yüksek boyutlu dosyalar işlendikten sonra cache'de tutulabilir.
  - İşletim sistemi bellek yetersiz kaldığında cache dizinlerini otomatik temizler (garbage collector kadar sık değil).
  - Normal dizinlerde tek tek dosya kontrolü gerekir, cache'de toplu silme yapılabilir.
- Content Provider
  - Uygulamalar arası veri paylaşımı için kullanılan yapı.
  - Rehber uygulaması, SMS mesajları gibi paylaşımlı verilere erişim sağlar.
  - Cursor yapısı ile veri okuma.
- Firebase
  - Google'ın cloud tabanlı veri saklama çözümü.
  - Karmaşık olmayan yapısı sayesinde verilerin sunucuda saklanması, uygulama yükünün azaltılması, veri kaybı riskinin minimize edilmesi.
  - Lokalde veri saklamak ve network üzerinde/cloud'da veri saklamak seçenekleri.

Hocanın Özellikle Vurguladığı Kısımlar

- Veri saklama seçiminde sorulması gereken kritik sorular
  - Ne kadarlık bir alana ihtiyaç var?
  - Verinin güvenlik seviyesi ne olmalı?
  - Veri özel mi yoksa paylaşılabilir mi?
  - Paylaşılabilir içerik ise shared storage, gizli ise internal storage + preferences + veritabanı + dosya.
  - Büyük boyuttaki oyun gibi veriler external storage'da saklanır.
- USB bağlantısı sırasında external storage uygulamalarının bloklanması
  - "Aklınızdan çıkarmayın" vurgusu; telefon USB'ye bağlandığında hard drive moduna geçer ve external storage uygulamaları kapatılır.
  - Bu durum sıklıkla göz ardı edilir ve uygulama beklenmedik şekilde kapanır.
- Cache kullanımının pil ömrüne etkisi
  - Cache dizinlerinin temizlenme sıklığı, dosya kontrolü yapmaya gerek olmaması pil ömrü için önemli.
  - Telefon açılıp kapatıldığında cache temizlenmesi konusunda kesin bilgisi olmadığını

belirtmiştir.
- Kodun kendiniz tarafından yazılmasının önemi
  - SQLite'ı anlatmamış olsa da bilen arkadaşların kullanmasına izin verdiği, ama bilmeyenler için zorunlu olmadığı.
  - Yapılan işin "kendiniz tarafından yapılmış olması"nın kritik olduğu; bir yerden alındıysa puan olarak değerlendirilmeyeceği.
- GitHub'da ödevin paylaşım zamanlaması
  - Teslim tarihinden önce public paylaşım yapılmaması gerektiği; kötüye kullanım riski.
  - Teslimden bir gün sonra private'ı public'e çevirme önerisi.

Kısa Tekrar Notları

- 4 veri saklama yöntemi: app-specific (internal/external), shared storage, preferences, veritabanı.
- App-specific: getFilesDir (kalıcı), getCacheDir (geçici).
- SharedPreferences: key-value formatında primitif veri + string.
- SQLite + Room: yapılandırılmış, ilişkisel veri.
- Firebase: cloud tabanlı veri saklama.
- USB'de external storage uygulamaları bloklanır.
- MANAGE_EXTERNAL_STORAGE: dosya bazında erişim (Android 10+).
- Scoped Storage: belli bölgelere erişim (Android 10+).
- FileOutputStream.write + close, FileInputStream.read.
- flash(): senkron yazma, aksi halde asenkron.
- Cache: pil ömrü için avantaj, otomatik temizlenir.

Detaylı Açıklamalar

Dersin başlangıcında hoca, sınav haftasının bittiğini, geçmiş olsun dileklerini iletmiştir. Sınav hakkında bir öğrenci sürenin az geldiğini, başka bir öğrenci soruların zor olmadığını belirtmiştir. Hoca final sınavında bir tık daha süreyi uzatma şansı olabileceğini söylemiştir.

İkinci ödev detaylı olarak anlatılmıştır. Ödevin tüm konuları kapsadığı, kullanıcı giriş/kayıt ekranı, menü ekranı, soru ekleme/listeleme, sınav ayar ekranı, sınav oluşturma ekranı gibi ekranlar içerdiği belirtilmiştir. Artık kullanıcı bilgilerinin kalıcı alana taşınması gerektiği (SQLite veya dosya), ArrayList'in yeterli olmadığı vurgulanmıştır. Soru ekleme ekranında 5 şık, doğru cevap, resim/ses/video eklenebilme özelliği; soru listeleme ekranında RecyclerView ile silme/güncelleme (silme sırasında dialog box ile onay); sınav ayar ekranında SharedPreferences ile süre/puan/zorluk düzeyi (2-5) saklama; sınav oluşturma ekranında metin formatında kaydetme ve mesajlaşma uygulaması üzerinden paylaşma özellikleri istenmiştir. Teslim şekli: kaynak kodları ve APK GitHub'a, 3-5 dakikalık YouTube videosu (kendini tanıtma + uygulamayı anlatma + neler yapılamadığını söyleme), PDF (link içeren) Online Yıldız Teknik Üniversitesi'ne yüklenecektir. Süre önümüzdeki hafta çarşamba gece yarısına kadardır. Hoca, ödevi teslim ederken dikkat edilmesi gereken önemli bir noktayı vurgulamıştır: GitHub'da ödevin teslim süresinden önce public şekilde paylaşılmaması gerektiği, aksi halde kötüye kullanılabileceği, ödevi teslim ettikten bir gün sonra private'ı public'e çevrilebileceği belirtilmiştir. UI puanı konusunda beklenti: tutarlı, dengeli, hızlıca her şeyin yerleştirilmediği, planlı arayüzler; yeni UI bileşenleri kullanmak ekstra puan kazandıracaktır.

Dersin ana konusuna, yani veri saklama yöntemlerine geçildiğinde dört temel yöntem tanıtılmıştır. App-specific storage (uygulamaya özel depolama) iki türlü olabilir: dahili bellek (internal storage) ve harici bellek (external storage). Shared storage ile video, fotoğraf, ses ve diğer belge türleri saklanabilir. Preferences ise uygulama ve oyun ayarları gibi basit primitif verileri key-value şeklinde saklar. Veritabanı (SQLite + Room) yapılandırılmış, ilişkisel veriler için kullanılır. Firebase ise cloud tabanlı veri saklama çözümüdür. Hoca, ne tür veri saklama yönteminin kullanılacağına karar vermek için şu soruların sorulması gerektiğini vurgulamıştır: ne kadarlık bir alana ihtiyaç var, verinin güvenlik seviyesi ne olmalı, veri özel mi yoksa paylaşılabilir mi. Paylaşılabilir içerik için shared storage, gizli veri için internal storage + preferences + veritabanı + dosya yapısı, büyük boyuttaki oyun gibi kritik olmayan veriler için external storage tercih edilmelidir.

App-specific storage detaylı olarak açıklanmıştır. Internal storage'da getFilesDir (kalıcı

bilgiler) ve getCacheDir (geçici bilgiler) dizinleri kullanılır. Harici bellek karşılığı ise getExternalFilesDir ve getExternalCacheDir'dir. App-specific dosyalar uygulama kaldırıldığında otomatik olarak silinir. Shared storage MediaStore API (medya dosyaları) ve Storage Access Framework (diğer dosyalar) üzerinden erişilebilir. İzinler: READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE ve Android 11 ile eklenen MANAGE_EXTERNAL_STORAGE. Android 10 ve 11 ile gelen Scoped Storage; belli bölgelere erişim yeteneği kazandırma, dosya bazında genel yönetim imkanı. Android 10 ile gelen internal storage'daki tüm verilerin şifrelenmesi özelliği, security seviyesini artırmıştır.

Internal ve external storage arasındaki fark vurgulanmıştır. Internal storage sürekli erişilebilir, external storage fiziksel olarak orada olduğu sürece erişilebilir. Internal storage'da harici alan yaratma imkanı vardır; fiziksel SD card olmak zorunda değildir, bazı firmalar internal storage üzerinde external bir alan yaratma imkanı verir. USB olarak Android telefon USB storage olarak enable edildiğinde external storage ile çalışan uygulamalar bloklanır ve kapatılır (telefon hard drive moduna geçer). Bu durumun "akıldan çıkarılmaması" gerektiği ısrarla vurgulanmıştır. External storage'a yüklenen uygulamaların APK'sı orada olsa da, uygulamaya özel dosyalar internal storage'da bulunur. Manifest'te installLocation özelliği ile uygulamanın external'a mı yoksa internal'a mı kurulacağı belirlenebilir (preferExternal, auto).

SharedPreferences kavramı açıklanmıştır. Primitif verileri (int, long, float, boolean) ve string'i key-value şeklinde saklar. XML dosyası kullanarak SharedPreferences API üzerinden bilgileri saklama imkanı vardır. Uygulama kaldırıldığında shared preferences dosyaları da silinir. Diğer uygulamalar tarafından erişilemez (güvenli alan). Tek bir veya birden fazla shared preferences dosyası oluşturulabilir. getSharedPreferences veya Activity'nin kendi getPreferences metodu kullanılabilir.

SQLite ve Room kavramı açıklanmıştır. Yapılandırılmış, ilişkisel veriler için internal storage'da veritabanı kullanılır. En alt katmanda SQLite, üst katmanda Room library ile erişim kolaylığı sağlanır. Room, SQLite'ın verimli kullanımını sağlar. Hoca, SQLite'ı anlatmamış olsa da bilen arkadaşların kullanmasına izin verdiğini, ama bilmeyenler için zorunlu olmadığını belirtmiştir. Dosya formatında saklama da kabul edilecektir. Yapılan işin "kendiniz tarafından yapılmış olması"nın kritik olduğu, bir yerden alındıysa puan olarak değerlendirilmeyeceği vurgulanmıştır.

Dosya okuma/yazma işlemleri kod üzerinden açıklanmıştır. getFileStream file objesi üzerinden dosya oluşturma, openFileOutput metodu context üzerinden doğrudan dosyaya erişim imkanı sağlar. FileOutputStream yazma, FileInputStream okuma için kullanılır. write, close, flash metotları mevcuttur. flash verilerin senkron olarak storage'a yazılmasını sağlar; çağrılmazsa asenkron olarak uygun zamanda yazılır. Try-catch mekanizması dosya yok, açılamaz gibi durumlar nedeniyle zorunludur. StreamReader ile satır satır okuma yapılabilir. Parcelable ve Serializable interface'leri objelerin bütün halinde binary formatta kaydedilmesini sağlar.

Cache kullanımı detaylı olarak ele alınmıştır. Cache dizini sonradan silinmesi sorun yaratmayacak, kalıcı olması gerekmeyen dosyalar için uygundur. İnternetten indirilen yüksek boyutlu dosyalar işlendikten sonra cache'de tutulabilir. İşletim sistemi bellek yetersiz kaldığında cache dizinlerini otomatik temizler (garbage collector kadar sık değildir, belli bir oranı doldurmuş olmanız gerekir). Normal dizinlerde tek tek dosya kontrolü gerekir, cache'de toplu silme yapılabilir. Bu, pil ömrü için avantaj sağlar.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.